

LLM API Hands-On Task

Alternative Example: Using Groq and Gemini APIs to Explain Cloud Computing

1. Objective

The purpose of this exercise is to understand how a Python application communicates with an LLM through an API. In this example, the application sends a prompt asking the model to explain cloud computing and prints the generated response.

2. API Communication in Simple Terms

An API acts as a communication bridge between a Python program and an online LLM service. The program sends a prompt, the LLM processes it, and the generated response is returned.

Program -> Request -> API -> LLM -> Response -> Program

3. Main Components of an LLM API Call

- API Key – authenticates the application.
- Endpoint – URL used to access the API.
- Headers – contain authentication and content-type information.
- Request Body – contains model and prompt information.
- Response – normally returned as JSON.

4. Setup

Install the requests package:

```
pip install requests
```

5. Practical A — Groq

Replace the placeholder with your own API key. Do not publish your API key.

```
import requests

api_key = "YOUR_GROQ_API_KEY"
endpoint = "https://api.groq.com/openai/v1/chat/completions"

headers = {
    "Authorization": "Bearer " + api_key,
    "Content-Type": "application/json"
}

payload = {
    "model": "openai/gpt-oss-20b",
    "messages": [
        {
            "role": "user",
            "content": "Explain cloud computing in simple words with 3 examples."
        }
    ]
}

r = requests.post(endpoint, headers=headers, json=payload)
```

```

if r.status_code == 200:
    output = r.json()
    print(output["choices"][0]["message"]["content"])
else:
    print("Request failed:", r.status_code)
    print(r.text)

```

6. Practical B — Gemini

The exact Gemini model name can vary according to currently available models. Replace `MODEL_NAME` with an available model.

```

import requests

api_key = "YOUR_GEMINI_API_KEY"
model = "MODEL_NAME"
endpoint = (
    "https://generativelanguage.googleapis.com/v1beta/models/"
    + model + ":generateContent"
)

headers = {
    "x-goog-api-key": api_key,
    "Content-Type": "application/json"
}

payload = {
    "contents": [
        {
            "parts": [
                {
                    "text": "Explain cloud computing in simple words with 3 examples."
                }
            ]
        }
    ]
}

r = requests.post(endpoint, headers=headers, json=payload)

if r.status_code == 200:
    output = r.json()
    print(output["candidates"][0]["content"]["parts"][0]["text"])
else:
    print("Request failed:", r.status_code)
    print(r.text)

```

7. Understanding the Code

`requests.post()` sends the HTTP POST request. `headers` provide authentication and identify JSON data. `json=payload` sends the request body as JSON. `response.json()` converts the response into Python data so the generated text can be extracted.

8. Example Prompt and Expected Output

Prompt: Explain cloud computing in simple words with 3 examples.

Cloud computing means using computing services over the Internet instead of depending only on your own computer.

- Google Drive – stores files online.
- Netflix – streams movies using online servers.
- Gmail – stores and manages emails online.

9. Experiments to Perform

Run the program first with the cloud-computing prompt. Then change only the prompt and observe how the response changes. Try these prompts:

- Explain Artificial Intelligence to a 10-year-old.
- Give three uses of machine learning.
- Create five Python interview questions with answers.
- Explain cybersecurity in simple words.

10. Error Handling

- 200 – Request succeeded.
- 400 – Request data is incorrect.
- 401 – Authentication/API key problem.
- 403 – Access is forbidden.
- 404 – Endpoint or resource was not found.
- 429 – Rate limit exceeded.
- 500 – Server-side error.

11. Viva Questions and Short Answers

1. Why is POST used?

Because prompt and other data must be sent to the API.

2. Why is JSON used?

It provides a structured way to represent request and response data.

3. What is an endpoint?

It is the URL through which an API service is accessed.

4. What is authentication?

It verifies that the application is allowed to use the API.

5. What is a status code?

It indicates whether the HTTP request succeeded or failed.

12. Conclusion

This practical demonstrates how an LLM API can be called directly from Python using requests. By changing the prompt, the same program can be used for many educational and application tasks. The exercise also demonstrates authentication, endpoints, headers, JSON payloads, POST requests, and response handling.